

-L dnsm read Line-oriented realtime score events from device 'dnsm'

You may use the option to send the score in real time by using the device *stdin*!

For the minimal application, you will have two widgets to set the instrument index and note number. You can then use a 'Play' button that will play the note as long as the button is pressed.

You can create a better version by replacing the note number text box by multiple buttons or map keys on the keyboard to note numbers.

First, you need to write the code to start the *csound* server listening on the *stdin*. So, write the following code in *tk_sf2.py*, which uses the subprocess module. It will start the *csound* command and open a pipe for *stdin*.

```
import subprocess
def start_csound():
    command = ['csound -dm0 -T -odac -rtaudio=alsa -L stdin sf2.csd']
    csd = subprocess.Popen(command, shell=True, stdin = subprocess
PIPE)
    return csd
```

You will need to add a line to the score section of *sf2.csd* as follows:

```
<CsScore>
i 1 0 180
</CsScore>
```

The score duration of 180 seconds ensures that *csound* will listen on *stdin* for three minutes before shutting down. There must be a better solution but I have not found one, so far.

Now, you can add the code to display a minimalist and very plain looking Tkinter form. The form contains a list box to select the instrument, a text entry widget for the MIDI note number and a button that calls the method *play_note* when pressed and *stop_note* when released. You can fill in the code in these methods shortly.

```
def play_form():
    def play_note(e):
        pass
    def stop_note(e):
        pass
    root = Tk()
    instr_box = Listbox(root)
    for instr in instruments:
        instr_box.insert(END, instr)
    instr_box.select_set(3)
    instr_box.pack()
    note_label=Label(root, text='Enter Note Number: 0-127')
    note_label.pack()
    note_box = Entry(root)
```

```
note_box.pack()
play = Button(root, text='Play')
play.bind('<Button-1>', play_note)
play.bind('<ButtonRelease-1>', stop_note)
play.pack()
return root
```

Now, add the code to start the *csound* server, fill in the instruments of interest in the list box and start the GUI application.

```
csd = start_csound()
instruments = {'Piano':0, 'Guitar':24, 'Violin':40, 'Sitar':104, 'Drums':117}
play_form().mainloop()
```

Finally, you will replace the dummy *play_note* and *stop_note* methods with appropriate code:

```
def play_note(e):
    notenum = note_box.get()
    instr = instr_box.get(instr_box.curselection()[0])
    preset_index = instruments[instr]
    # i line: parameters p1. p7
    # p1. p2. p3 : csound instrument, starting time, duration,
    # p4. p7: velocity, Midi Note Num, Preset Index, Amp
    csd.stdin.write('\1 0 -1 127 ' + notenum + ' ' + str(preset_index) + '
2\n')

def stop_note(e):
    csd.stdin.write('\1 0 0 \n')
```

To play a note, you should pick up the preset index and note number. Then you need to construct the string and write it on the *stdin* pipe of *csound*. A value of -1 for the duration implies that you keep playing the note till another score command is received for the instrument. Since it is being played in realtime, the start time indicates the amount of delay after this score line has been received.

So, the *stop_note* method is simple. It just writes a score to play the piano for '0' period of time with '0' velocity; hence, stopping the note which was being played.

This leads to the question of how to create an orchestra of multiple instruments? You will need to modify *sf2.csd* file to define additional instruments like the first one. Then, independent score commands can be given to each of the instruments.

Now, imagine programming each of the many instruments and, then imagine the delightful effect when all these instruments play together. No wonder great composers are a rarity! 

By: Dr. Anil Seth

The author is a consultant by profession and can be reached at seth.anil@gmail.com